LIMITBEYOND

FITNESS & GYM MANAGEMENT PLATFORM

System Architecture & Technology Stack

Version 1.0 · How LB-SRS-1.1 will be built

Document ID	**LB-ARCH-1.0**
Status	**Draft for review**
Date	**16 September 2026**
Prepared by	**Bibek Jha**
Implements	**LB-SRS-1.1, 16 Sep 2026**
Stack	**Java 21 · Spring Boot · PostgreSQL · React Native (Expo)**
Deployment	**AWS EC2 (Docker) · Neon Postgres · Cloudinary**

> **Scope of this document.** The SRS fixed *what* the system must do. This document fixes *how* — the technology choices, the shape of the codebase, the data and security architecture, and the deployment topology. Every significant choice is recorded as an ADR with its alternatives and the conditions under which it should be revisited, so that a decision made today can be re-examined on evidence rather than re-argued from memory.

Table of Contents

1. Introduction and Approach

1.1 Purpose · 1.2 Relationship to the SRS · 1.3 Guiding Principles · 1.4 Conventions

2. The Stack at a Glance

3. Architecture Decision Records (ADR-01 – ADR-15)

Including ADR-05, the rejected server-side write buffer; ADR-07, the database region trade-off; and ADR-15, media handling and its privacy constraint

4. System Architecture

4.1 Context · 4.2 The Modular Monolith · 4.3 Module Map · 4.4 Request Lifecycle

5. Data Architecture

5.1 Principles · 5.2 Core Schema · 5.3 Row-Level Security · 5.4 Migrations · 5.5 Indexing

6. Security Architecture

6.1 Authentication · 6.2 Authorisation · 6.3 Isolation Enforcement · 6.4 Secrets and Media

7. Mobile Architecture

7.1 Structure · 7.2 Offline Sync · 7.3 Idempotency · 7.4 Push Notifications

8. Background Processing

9. Integration Architecture

10. Deployment, Environments and CI/CD

11. Observability, Backup and Recovery

12. Performance and Scaling Strategy

13. Indicative Cost Model

14. Build Sequence by Phase

15. Risks and Open Decisions

16. Appendix A — SRS Requirement to Architecture Mapping

17. Appendix B — SRS Amendments Required

1. Introduction and Approach

1.1 Purpose

This document specifies the technical architecture for LIMITBEYOND: the technology stack, the internal structure of the codebase, the data and security design, the mobile application architecture, and how the system is deployed and operated. It is the direct successor to LB-SRS-1.1 and assumes its requirements as given.

It is written to be actionable. Where a choice has a non-obvious implementation consequence — connection pooling interacting with row-level security, say, or idempotency keys being what makes offline sync safe — the consequence is stated here rather than left to be discovered during the build.

1.2 Relationship to the SRS

The SRS defines 21 functional modules and roughly 200 numbered requirements. Three of its requirements shape this architecture more than the rest, and each is answered by a specific decision recorded below:

Requirement	What it demands	Architectural answer
NFR-SEC-01a	A Location Admin must never reach a sibling location's data, by any interface including exports and identifier guessing.	PostgreSQL Row-Level Security (ADR-04). Isolation is a database policy, not a convention the application is trusted to follow.
NFR-PRF-05	Class booking must never oversell a session under concurrent demand.	A single transactional authority — one Postgres database, one `@Transactional` boundary, a unique constraint doing the enforcing (ADR-03, ADR-05).
FR-ATT-07	A check-in attempted without connectivity must be queued and submitted on reconnection.	Client-side SQLite outbox with idempotency keys (ADR-06), on the device where the connectivity problem actually is.

1.3 Guiding Principles

ID	Principle	What it means in practice
PRN-01	Correctness before speed	Bookings, credits, payments and receipts are money and trust. Where the two conflict, transactional correctness wins — and at this scale the conflict is almost always imaginary, because the database is never the bottleneck (see ADR-05).
PRN-02	Enforce at the lowest layer	A rule enforced in the database cannot be forgotten by an endpoint. A rule enforced in a shared filter cannot be forgotten by a controller. Push every invariant as far down as it will go.
PRN-03	Boring technology, deliberately	One language, one database, one deployable. Novelty is spent only where the problem genuinely demands it. A solo or small team's scarcest resource is attention, not compute.
PRN-04	Defer distribution	Nothing is split into a separate service until a specific, measured pressure requires it. Distributed systems are a cost paid daily for a benefit realised rarely.
PRN-05	Every external service is optional	SMS, push, WhatsApp and AI providers can each be unavailable without core operations failing. This is a requirement (NFR-AVL-04) and an architectural constraint on how integrations are written.

ID	Principle	What it means in practice
PRN-06	Decisions carry expiry conditions	Every ADR names what would make it wrong. A choice appropriate for a development phase is recorded as such, with the trigger for revisiting it, rather than silently hardening into permanence.

1.4 Conventions

Decisions are recorded as Architecture Decision Records: **ADR-nn**, each with context, the decision, the alternatives considered, consequences, and a revisit trigger. An ADR marked *Rejected* records an option that was considered and declined, with the reasoning — these are as valuable as the accepted ones, because they stop settled questions being reopened without new evidence.

Requirement identifiers in the form `FR–XXX–nn` and `NFR–XXX–nn` refer to LB-SRS-1.1.

2. The Stack at a Glance

Layer	Choice	Rationale, in one line	ADR
Mobile	React Native + Expo (SDK 52+), TypeScript	One codebase for iOS and Android; EAS handles builds, store submission and over-the-air updates.	—
Mobile state	TanStack Query + Zustand	Server state cached and invalidated by Query; the little genuinely-local state in Zustand. Carried over from the predecessor's web app.	13
Mobile local DB	expo-sqlite + Drizzle ORM	Offline outbox for check-in and workout logging — the two things done on a gym floor with no signal.	06
Web console	React + Vite + TypeScript + shadcn/ui	Same component vocabulary as the predecessor, so existing UI work is reusable.	—
Backend	Java 21, Spring Boot 3.3+, Maven	Proven in limitbeyond2.0 and known to the team; virtual threads suit bursty check-in load.	02
Structure	Modular monolith (Spring Modulith)	21 SRS modules with enforced boundaries, one deployable, one transaction scope.	01
API	REST + OpenAPI 3 (springdoc)	Generated, type-safe TypeScript clients recover most of what a shared-language backend would have given.	13
Database	PostgreSQL 16	Relational domain, transactional integrity, and Row-Level Security answering the isolation requirement directly.	03
DB hosting	Neon (development) → RDS (production)	Cheap with free credits for now; region latency makes this explicitly provisional.	07
Isolation	Row-Level Security policies	Gym and location isolation enforced by the database on every query.	04
Migrations	Flyway	Versioned SQL, applied by a privileged user distinct from the application's runtime user.	04
Auth	Spring Security + JWT, Twilio Verify for OTP	Bespoke role × location model no managed product expresses; Verify covers phone and email in one API.	08
Jobs	Postgres queue, `SKIP LOCKED`	No Redis, no SQS. The database you already run is a competent queue at this scale.	09
Push	Expo Push Notification Service	Wraps FCM and APNs; delivery receipts satisfy FR-NTF-06. No WebSocket tier in any phase.	10
Media	Cloudinary (development)	Storage, CDN and on-the-fly transformation in one service. Sensitive media must use authenticated delivery, not the default public type — see ADR-15.	15
Compute	AWS EC2, Docker, Nginx + Let's Encrypt	Single instance for development; ALB and a second instance when traffic justifies it.	12
CI/CD	GitHub Actions + Expo EAS	Image build and deploy on the backend; EAS Build and Submit for the stores.	14
Observability	Sentry, Micrometer, structured JSON logs	Diagnose a failed check-in or booking without reproducing it (NFR-MNT-02).	14

Layer	Choice	Rationale, in one line	ADR
Testing	JUnit 5, Testcontainers, REST Assured	Testcontainers runs real Postgres, which is the only way to genuinely test RLS policies.	04

3. Architecture Decision Records

ADR-01 · Modular monolith, not microservices — ACCEPTED

Context	The SRS defines 21 modules. The instinct to map modules onto services is strong and, here, wrong.
Decision	One Spring Boot application, one database, one deployable, with module boundaries enforced at build time by Spring Modulith. Modules communicate through published interfaces and in-process events, never by reaching into each other's internals.
Why	Booking a class touches session capacity, the member's entitlement, their class-pack balance and the notification queue. In a monolith that is one transaction and it either happens or it doesn't. Across services it is a distributed transaction, a saga, and a compensating-action path for every failure mode — for a small team, with no operational platform, and no scaling pressure that would justify it.
Consequences	Deployment is trivial. Refactoring across module boundaries stays cheap. The whole application scales as a unit, and a build failure blocks everything — acceptable at this size.
Revisit if	One module develops a genuinely different scaling profile (AI inference is the likeliest candidate), or the team grows past the point where a single codebase causes contention.

ADR-02 · Java 21 and Spring Boot for the backend — ACCEPTED

Decision	Java 21 (LTS) with Spring Boot 3.3+, built with Maven, as in limitbeyond2.0.
Why	Three reasons in order of weight. The team already knows it and has it working. `@Transactional` is precisely the tool the booking, credit-decrement and payment requirements need. Spring Security is flexible enough to express the role × location authorisation model, which is bespoke enough that most frameworks would fight it.
Alternatives	*NestJS / TypeScript* — would share a language with React Native, a real benefit, recovered here instead by generating TypeScript clients from OpenAPI (ADR-13). Not worth discarding existing expertise. *Go* — leaner runtime, faster start, but a smaller ecosystem for this domain and a new language to learn while also learning the domain.
Consequences	A JVM baseline of roughly 300–500MB constrains instance sizing. Cold start is measured in seconds, so the deployment model must keep an instance warm — which it does. Java 21 virtual threads let the bursty 6am check-in load be handled with ordinary blocking code, no reactive programming required.
Revisit if	Never, absent a team composition change. This is a settled decision.

ADR-03 · PostgreSQL as the single system of record — ACCEPTED

Decision	PostgreSQL 16 holds all operational data. The predecessor's MongoDB is not carried forward.
Why	The domain is relational to its core — role grants against locations, bookings against sessions against capacity, credits drawn against packages, payments allocated against memberships. Every one of these is a constraint the database can enforce, and every one of them is a bug waiting to happen if it cannot. Row-Level Security additionally provides a direct answer to the hardest security requirement (ADR-04), which no document store offers.
Where flexibility is needed	`JSONB` columns carry genuinely variable structures — health screening answers, AI generation payloads, notification template variables — giving document-style flexibility exactly where it earns its place, without giving up constraints everywhere else.
Consequences	Schema changes require migrations, which is a feature: the schema is reviewed, versioned and reversible. Nothing migrates from MongoDB (ADR-11), so there is no dual-model period.

ADR-04 · Row-Level Security for gym and location isolation — ACCEPTED

Context	`NFR-SEC-01a` requires that a Location Admin cannot reach a sibling location's data by *any* interface. `RSK-06` names cross-location leakage the most damaging possible defect and directs that isolation be enforced at the data access layer rather than per endpoint.
Decision	Isolation is a set of Postgres RLS policies. Request context is injected per transaction; every query is filtered by the database regardless of what the application code says.
Why	Application-layer scoping is only ever as reliable as the last developer's discipline. One endpoint written in a hurry, one repository method that forgets its filter, and a gym sees another gym's members. RLS makes that class of bug structurally impossible rather than merely discouraged.
Consequences	Two critical implementation constraints follow, detailed in §5.3: the application's database user must not own the tables, and context must be set with `SET LOCAL` inside a transaction. Getting either wrong silently disables the protection.
Verification	A standing Testcontainers suite attempts cross-location and cross-gym access through every repository and asserts empty results. This suite is the evidence for `NFR-SEC-01a` and must never be skipped.

ADR-05 · Server-side SQLite write buffer in front of Postgres — REJECTED

Proposal	Accept complex `PUT` / `POST` requests into a server-local SQLite file, acknowledge immediately, and synchronise to Postgres in the background — for speed on write-heavy operations.
Decision	Rejected. Recorded here in full because the underlying instinct is sound and will recur; it is the tier that is wrong, not the idea.
The arithmetic	`NFR-PRF-02` budgets 2000ms for an end-to-end check-in on a 3G connection. Of that, mobile network round trip consumes 100–300ms, serialisation 5–20ms, authorisation 1–5ms, business logic 1–10ms — and the Postgres insert 1–5ms. The database write is roughly **0.25% of the budget**. Substituting SQLite saves single-digit milliseconds on a quarter-second request.
What it costs	**(1)** Horizontal scaling breaks immediately — two instances hold two SQLite files, so a write to one is invisible to the other and read-after-write consistency is lost. **(2)** `NFR-PRF-05` becomes unsatisfiable: capacity cannot be enforced across independent files, converting a solved transactional problem into an unsolved consensus one. The same applies to `FR-BIL-10` credit decrements, `FR-ATT-05` duplicate rejection, and `FR-BIL-05` gapless receipt numbering. **(3)** Acknowledging before durable commit is a correctness lie on ephemeral container storage — the instance dies, the payment is gone, and the admin was told it saved. **(4)** It introduces a bespoke sync engine, conflict resolution, replay, dead-lettering and lag monitoring as permanent maintenance.
What was adopted instead	The valid form of the idea, in two places. *On the device* — SQLite as an offline outbox where the latency and connectivity problems genuinely are (ADR-06). *On the server* — commit to Postgres transactionally in single-digit milliseconds, then defer the expensive derived work (streak recalculation, notification dispatch, analytics rollup) to a job queue (ADR-09). This yields the perceived speed without surrendering any guarantee.
Revisit if	Measurement shows database write latency materially contributing to p95 response time. It will not at 100,000 members; the correct response then would be read replicas or partitioning, still not a write buffer.

ADR-06 · Client-side SQLite outbox for check-in and workout logging — ACCEPTED

Decision	The mobile application holds a local SQLite database (`expo-sqlite` with Drizzle). Check-ins and workout set logs are written locally first, confirmed in the UI immediately, and synchronised to the server by a background task. No other operation is offline-capable.
Why these two	They share a property nothing else does: **they cannot conflict**. A check-in is an append-only fact about one person at one moment. A workout set log is owned exclusively by the person who created it. Neither requires arbitration against another user's concurrent action, so neither needs conflict resolution — the hard part of offline sync simply does not arise. Bookings and payments are excluded precisely because they *do* contend, and must be arbitrated server-side.
Why it matters	Gym floors and basements have poor signal, and these are the two things members do there. Local-first write means the UI confirms in under 16ms regardless of network, which is the actual perceived-speed win the server-side buffer was reaching for.
Consequences	Every queued operation carries a client-generated idempotency key; the server deduplicates on it, which is also how `FR-ATT-05` is satisfied under retry. Deferred check-ins retain their original timestamp and are flagged as such, per `FR-ATT-07`.

ADR-07 · Neon Postgres for development, co-located in Singapore — PROVISIONAL

Decision	Neon serverless Postgres during development, for its cost and free credits. **Both EC2 and Neon are placed in `ap-southeast-1` (Singapore).**
The problem this avoids	Neon offers no Mumbai region; its nearest AWS region to India is Singapore. Placing EC2 in Mumbai and the database in Singapore puts 50–70ms of ocean between the application and its data — on *every query*. A request issuing five queries pays 250–350ms in network alone, breaching the 400ms p95 of `NFR-PRF-01` before any code executes.
Why co-location wins	Placing both in Singapore moves the crossing from the database path to the client path. An Indian user then pays roughly 50ms extra on **one** hop (phone to server) rather than 60ms on **five** (server to database, per query). One crossing beats five, and the database round trip returns to sub-millisecond.
Implementation notes	Use Neon's *pooled* connection endpoint, which runs PgBouncer in transaction mode. This is compatible with the `SET LOCAL` pattern RLS requires (§5.3) but not with session-level prepared statements — set `prepareThreshold=0` on the JDBC URL. Disable scale-to-zero on any environment where latency is measured, or the first request after idle pays a cold start of 500ms to several seconds.
Revisit when	Any of: the first paying gym goes live; sustained load makes Neon's pricing exceed RDS; or India data-residency becomes a contractual requirement. The target then is **RDS Postgres in `ap-south-1` (Mumbai)** with EC2 alongside it, which is the lowest-latency configuration for an Indian user base. Because both are stock PostgreSQL 16, that move is a dump, restore and connection-string change — no application code is affected, which is precisely why this decision is safe to defer.

ADR-08 · Self-built JWT authentication with Twilio Verify for OTP — ACCEPTED

Decision	Spring Security issues and validates JWTs, as in limitbeyond2.0. One-time passwords for both phone and email are delivered through the **Twilio Verify** API.
Why not a managed identity product	The authorisation model — a set of role grants each scoped to a gym and a list of locations, feeding RLS context on every request — is bespoke. No managed product expresses it, so it would live in our database regardless, leaving a managed service to handle only the easy half while introducing a second source of truth about users to keep synchronised.
Why Verify rather than raw SMS	Verify handles code generation, expiry, retry limits and channel fallback as a service, which is meaningful security logic not worth reimplementing. It covers SMS and email through one API — matching the requirement for both — and can later add WhatsApp as an OTP channel on the same integration.
Tokens	Access tokens are short-lived (15 minutes) and carry the user's role grants; refresh tokens are long-lived, stored server-side and revocable, satisfying `NFR-SEC-04` and `FR-AUTH-13`. Revocation on suspension is immediate for refresh, and bounded by access-token lifetime otherwise.
Development constraint	A single verified device is used for OTP throughout development. This fits a Twilio trial account, which can only deliver to pre-verified numbers, and keeps development cost at zero. The consequence to plan around: **multi-user flows cannot be exercised over real OTP** — verifying a trainer while signed in as an admin, or testing two members booking the same last class slot, needs more than one account. Accordingly, non-production environments expose a seeded test-user path that bypasses OTP delivery, disabled by configuration in production and covered by an automated assertion that it is off.
Dependency with lead time	Transactional SMS into India requires **DLT registration** under TRAI rules — sender ID and message templates registered before a single OTP can be delivered. Typically one to two weeks. Development proceeds without it on the single verified device, but **no pilot with real members can start until it completes**, so begin the registration early rather than at the end of Phase 1.

ADR-09 · Background jobs on a Postgres-backed queue — ACCEPTED

Decision	Deferred work runs from a `job_queue` table consumed with `SELECT … FOR UPDATE SKIP LOCKED`. No Redis, no SQS, no broker.
Why	The workload is notification dispatch, expiry sweeps, streak recalculation, WhatsApp sends and analytics rollups — thousands of jobs per day, not millions per hour. Postgres handles this comfortably, and jobs enqueued in the same transaction as the business change they follow cannot be orphaned by a rollback. That transactional coupling is a genuine correctness benefit an external broker cannot offer.
Consequences	Retries with exponential backoff, a dead-letter state after a configured attempt count, and job records retained for inspection. Scheduled sweeps run under `ShedLock` so that a second instance does not duplicate them.
Revisit if	Sustained job throughput exceeds a few hundred per second, or a workload needs fan-out to multiple independent consumers.

ADR-10 · Expo Push for notifications; no realtime tier — ACCEPTED

Decision	Notifications are delivered by the Expo Push Notification Service, which wraps FCM and APNs. **No WebSocket or SSE infrastructure is built in any phase.**
Why no realtime	Examining the SRS for genuine realtime needs finds none. Waitlist promotion (`FR-CLS-08`) is a push notification. Session reminders are scheduled push. A trainer's class roster is refreshed by pull-to-refresh or a short poll while the screen is open. Not one requirement needs a server-initiated stream to an open connection — so a whole tier of infrastructure, and its reconnection and scaling problems, is avoided by simply noticing that.
Consequences	Expo's receipts endpoint gives per-message delivery status, satisfying `FR-NTF-06`. FCM credentials and an APNs key are configured through EAS. Expo's service is a dependency, but per `PRN-05` a push failure degrades only notification delivery — the in-application inbox remains the authoritative record.

ADR-11 · Clean start; nothing migrated from limitbeyond2.0 — ACCEPTED

Decision	No data is migrated. The exercise library is **authored fresh** against the richer model the SRS specifies — primary and secondary muscle groups, equipment, difficulty, instructions and media — rather than ported from MongoDB.
Why	The predecessor's exercise records lack the secondary muscle group, equipment and difficulty fields the new model requires, so a port would produce incomplete rows needing manual completion anyway. Authoring fresh is comparable effort and yields a clean, complete catalogue. Members, workouts, payments and history start empty; the predecessor continues running until cutover.
Consequences	This closes `OPN-02`. It also contradicts `FR-EXR-05`, which requires the predecessor's taxonomy and seed data be carried across — see Appendix B for the SRS amendment. Seed data ships as Flyway migrations so the catalogue is versioned and reproducible across environments.

ADR-12 · EC2 with Docker — ACCEPTED

Decision	The backend runs as a Docker image on a single EC2 instance (`t4g.small` or `t4g.medium`, ARM Graviton) behind Nginx terminating TLS via Let's Encrypt.
Why	Cheapest credible production-shaped topology, and the same Docker image runs unchanged on ECS or App Runner later. Nginx rather than an ALB saves roughly ₹1,400/month during development, at the cost of managing certificates — which certbot automates.
Media	Media is *not* stored on the instance or in S3; see ADR-15. Keeping image bytes off EC2 entirely matters on a 2GB instance.
Scaling path	Vertical first — instance size is a one-line change. Then an ALB with two instances, at which point sticky sessions are unnecessary because the application is stateless and the job queue already tolerates multiple consumers.

ADR-13 · REST with OpenAPI-generated clients — ACCEPTED

Decision	A versioned REST API (`/api/v1`) documented by springdoc-generated OpenAPI 3. TypeScript clients and types are generated from that specification into both the mobile app and the web console.
Why	Generation recovers most of the type-safety benefit a TypeScript backend would have offered: the contract is single-sourced, and a breaking backend change fails the client build rather than surfacing as a runtime error in a member's hands. Versioning the path satisfies `NFR-MNT-04`, which matters more than usual here — app-store review latency means old app builds stay in circulation for weeks.
Alternative	GraphQL was considered and declined: it would complicate RLS context propagation and per-field authorisation for no benefit the mobile client actually needs.

ADR-14 · GitHub Actions and Expo EAS for delivery — ACCEPTED

Decision	GitHub Actions builds, tests and deploys the backend image. Expo EAS builds and submits the mobile application, with EAS Update for over-the-air JavaScript changes.
Why OTA matters here	Apple review takes one to three days. Without over-the-air updates, a one-line JavaScript bug fix waits on that queue. EAS Update ships JS-only changes directly, reserving store submission for changes that touch native code. This materially changes how quickly production problems can be corrected.
Store constraints to plan for	An Apple Developer account ($99/year) and Google Play registration ($25 once). Apple requires a published privacy policy and a clear in-context rationale for each permission — directly relevant to health screening (`FR-MEM-03`) and location access for geofenced check-in (`FR-ATT-02`). Apps handling health-adjacent data receive closer review; the SRS position that this is fitness guidance and not medical advice (`CON-07`, `FR-NUT-10`) should be reflected in the store listing.

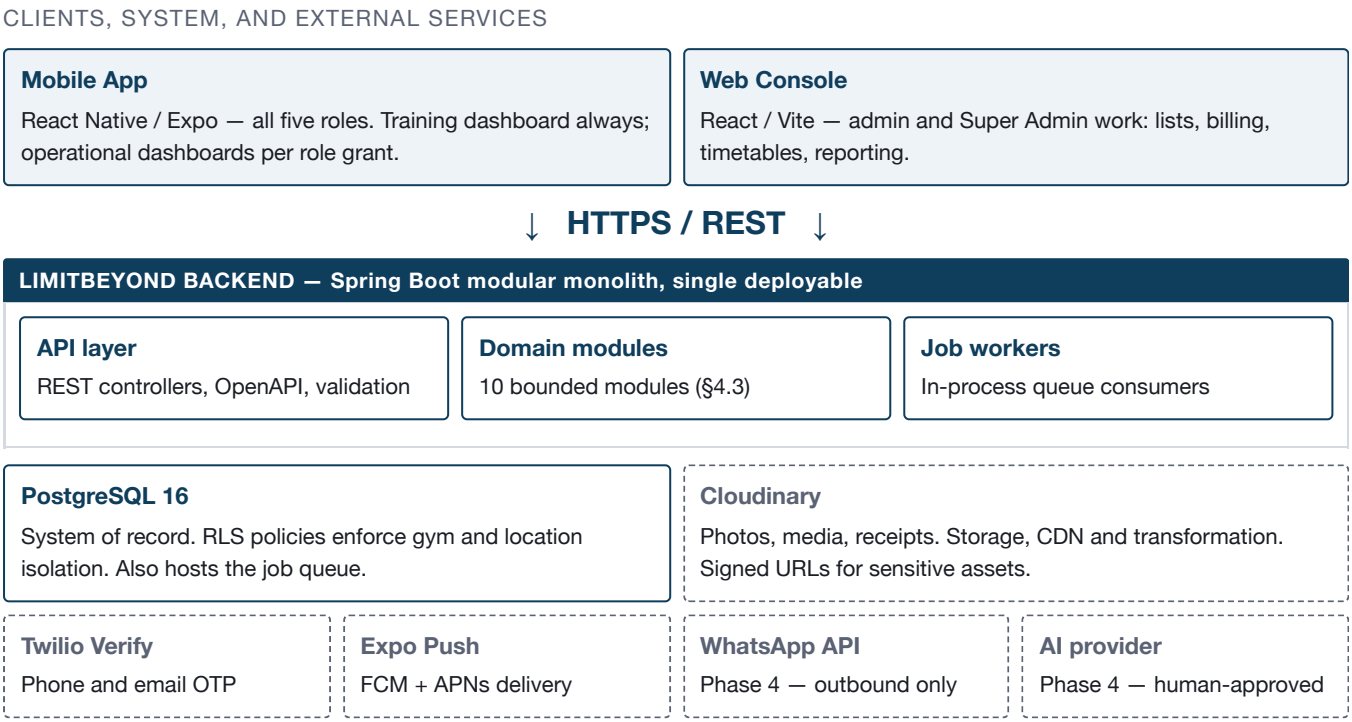
ADR-15 · Cloudinary for media, with authenticated delivery for sensitive assets

PROVISIONAL

Decision	Cloudinary stores and delivers all media during development — profile images, exercise demonstration media, generated receipts and optional identity documents — on its free tier.
Why it fits	Cloudinary is not merely storage, and the difference is what justifies it. **Transformations are URL parameters**, so the thumbnail and rendition pipeline that S3 would require you to build (Lambda plus an image library, invalidation, a variants table) simply does not need to exist. **`f_auto,q_auto` serves WebP or AVIF adaptively** per device, which directly serves `CM-04`'s requirement to work on 3G-class connections — payloads shrink substantially with no application code. A CDN is included, so no CloudFront distribution is configured. And signed direct-from-client upload keeps image bytes off EC2 entirely, which is not a small thing on a 2GB instance.
The constraint that matters	**Cloudinary's default `upload` delivery type produces permanent, public URLs.** Applied naively to a receipt or an identity document, that breaches `NFR-SEC-07` ("never reachable by unauthenticated URL") and `NFR-PRV-02` (these are sensitive personal data). A public URL, once leaked, is permanent and unrevocable.
Required handling	Media is split by sensitivity, and the split is enforced in the upload path rather than left to the caller's judgement: • **Sensitive** — generated receipts and optional identity documents. Uploaded as `type: authenticated` or `type: private`, delivered *only* through signed, time-limited URLs generated server-side after an authorisation check. Never public. • **Public** — gym logos, exercise demonstration images, generic app assets. Default `upload` type, fully CDN-cached. These are not sensitive and benefit from aggressive caching.
Verify before Phase 2	Time-limited delivery authentication may be gated to higher Cloudinary plans. **Establish which signing mechanism the account actually supports before receipts ship in Phase 1**, not after. If expiring authenticated delivery proves unavailable on the plan in use, route sensitive media to S3 presigned URLs (the original ADR-12 design, unchanged and still valid) and keep Cloudinary for public assets only. The abstraction below makes that a configuration change.
Lock-in rule	**Store only the Cloudinary `public_id` in the database — never a full URL.** Every URL is constructed at read time by a `MediaService` abstraction that also applies the sensitivity rules above. This matters more here than it did for the database: Neon to RDS is a dump and restore because both are stock Postgres, whereas a media provider swap has no equivalent. Without this rule, provider URLs embed themselves in every row referencing an image and the migration becomes a data rewrite. With it, it is one class.
Revisit when	Free-tier credits are exhausted — note that Cloudinary credits are consumed by storage, bandwidth *and* transformations together, so exercise media and receipts at real volumes will pass the free tier well before storage alone would suggest. Or when sensitive-media authentication proves plan-gated. The fallback in either case is S3 with presigned URLs for private media, and the abstraction is what makes that cheap.

4. System Architecture

4.1 System Context



Dashed boxes are external services. Per `PRN-05` and `NFR-AVL-04`, each may be unavailable without core operations failing: sign-in falls back to email and password, notifications persist in the in-app inbox, WhatsApp falls back to push, and AI features degrade to manual authoring.

4.2 Why a Monolith Is the Right Shape Here

The instinct to decompose 21 modules into services is worth confronting directly, because the domain contains an operation that settles it. Consider booking the last place in a yoga class, which `NFR-PRF-05` says must never oversell:

```
// Modular monolith — one transaction, one database, one outcome
@Transactional
public Booking book(SessionId sessionId, UserId userId) {
    Session s = sessions.lockForUpdate(sessionId);     // row lock held
    if (s.bookedCount() >= s.capacity())
        return waitlist.add(sessionId, userId);        // same transaction
    entitlements.consume(userId, s);                   // same transaction
    Booking b = bookings.create(sessionId, userId);    // same transaction
    jobs.enqueue(new BookingConfirmed(b.id()));        // same transaction
    return b;                                          // commit: all of it, or none of it
}
```

Split across a scheduling service, an entitlements service and a notifications service, that same operation becomes a distributed saga: a booking reserved in one service, an entitlement consumed in another, a compensating release if the second call fails, and an unresolved window in which the two disagree. Every failure path must be designed, implemented and tested. The monolith gets the same guarantee from a database transaction that has been correct since before any of us started working.

Module boundaries are still enforced — Spring Modulith fails the build when one module reaches into another's internals — so the structure that would permit later extraction is preserved without paying for distribution now.

4.3 Module Map

The SRS's 21 functional modules collapse into 10 code modules, grouped by what changes together rather than by SRS numbering.

Module	SRS	Responsibility
platform	M20	Gym lifecycle, subscription offerings, cash payment records, activation, lapse and suspension. Super Admin operations.
identity	M1, M19	Authentication, OTP, tokens, role grants and their location scope, promotion and demotion, permission evaluation, audit trail, consent.
org	M2	Gym profiles, locations, geo-coordinates and radii, operating hours, closure calendars, tenant configuration.
people	M3, M4	Member and trainer profiles, health screening, specialisations, certifications, availability, trainer-to-member assignment.
billing	M5	Plan catalogue, memberships, PT packages and credits, payment records, dues, receipts with GST, freezes, refunds.
attendance	M6	QR and geofence check-in, manual marking, purpose (shift versus personal workout), idempotent ingestion, streaks.
scheduling	M7, M8, M9	Class types, recurring timetables, session generation, bookings, capacity, waitlists, PT appointments, online meeting links.
training	M10–M13, M21	Exercise catalogue, workout templates and routines, set logging, personal records, nutrition plans, progress tracking — and the personal fitness profile every account holds.
engagement	M14, M15, M17	Feedback in both directions, coaching notes, announcements, the notification catalogue and preferences, WhatsApp dispatch.
insights	M16, M18	Dashboards and reports scoped by location, exports, churn worklists, AI generation with human approval.

> **Note on `training`.** It deliberately serves every role, not only members — that is the architectural expression of `FR-PFP-01`. No method in this module accepts a role as a parameter or branches on one. A Gym Owner logging their own sets travels exactly the same code path as a member, which is the most reliable way to guarantee the SRS's dual-facet model survives implementation.

4.4 Request Lifecycle — a geofenced check-in

Following one request end to end shows where the time goes and where each guarantee is enforced. Timings assume EC2 and Postgres co-located per ADR-07.

1	**Mobile app**	Writes the check-in to local SQLite with a generated idempotency key; the UI confirms immediately. The user's experience of "instant" ends here — everything following is invisible to them.	~8ms
2	**Network**	Background sync task posts the queued entry. This is the dominant cost and the one least under our control.	100–300ms
3	**Nginx**	TLS termination, proxy to the application.	~1ms
4	**Security filter**	JWT validated, role grants and location scope resolved into a request-scoped `AuthContext`.	1–3ms
5	**Transaction**	Transaction opens; `SET LOCAL` injects gym, location and role into the session. From here, RLS filters every query automatically.	<1ms
6	**attendance**	Idempotency key checked — a replay returns the original result rather than a duplicate. Membership validity checked (`FR-ATT-06`), geofence distance verified against the location's radius, duplicate window enforced (`FR-ATT-05`).	2–4ms
7	**Postgres**	Attendance row inserted with purpose and method; a `StreakRecalc` and any notification jobs are enqueued *in the same transaction*, so they cannot outlive a rollback.	2–5ms
8	**Commit**	Transaction commits. `SET LOCAL` context is discarded automatically with it — which is precisely why the connection can safely return to the pool.	1–3ms
9	**Worker**	Asynchronously, off the request path: streak recalculated, push dispatched, analytics rolled up.	async
	Server total	Against a 2000ms budget (`NFR-PRF-02`) — the server is 0.5% of it, the network 95%.	**7–17ms**

This is the concrete evidence behind ADR-05. There is no meaningful latency to reclaim on the server; the only optimisation that matters to a member is step 1, which is why the local-first write was adopted and the server-side buffer was not.

5. Data Architecture

5.1 Principles

- **Every row carries `gym_id`, and every operational row carries `location_id`** — satisfying `CON-01` and `FR-ORG-10`, and providing the columns RLS policies filter on.
- **Money and history are append-only.** Payments, attendance and audit entries are never updated or deleted; corrections are new rows referencing the original (`FR-BIL-12`, `FR-ADM-03`).
- **Constraints live in the database.** Capacity, uniqueness, credit non-negativity and referential integrity are enforced by Postgres, not merely by application code.
- **Identifiers are UUIDv7** — unguessable in URLs, while remaining time-ordered so index locality does not suffer as random UUIDs would.
- **Time is stored as `timestamptz`, always UTC,** rendered in the tenant's timezone at the edge (`NFR-USA-04`).

5.2 Core Schema Sketch

Abbreviated to the structurally interesting parts — the ones that encode an SRS rule rather than merely storing a field.

```sql
-- Authority is a grant, never a column on the user (FR-AUTH-11, FR-AUTH-26)
CREATE TABLE role_grants (
  id           uuid PRIMARY KEY,
  user_id      uuid NOT NULL REFERENCES users(id),
  gym_id       uuid NOT NULL REFERENCES gyms(id),
  role         role_type NOT NULL,        -- SUPER_ADMIN | GYM_OWNER | LOCATION_ADMIN | TRAINER |
MEMBER
  location_ids uuid[] NOT NULL DEFAULT '{}',   -- empty = gym-wide (GYM_OWNER only)
  granted_by   uuid REFERENCES users(id),      -- the delegation chain, FR-AUTH-25
  granted_at   timestamptz NOT NULL DEFAULT now(),
  revoked_at   timestamptz,
  -- A user holds at most one live grant of a given role per gym
  CONSTRAINT one_live_grant UNIQUE NULLS NOT DISTINCT (user_id, gym_id, role, revoked_at)
);
-- Only a Gym Owner may hold gym-wide scope (FR-AUTH-18)
ALTER TABLE role_grants ADD CONSTRAINT scope_requires_owner
  CHECK (cardinality(location_ids) > 0 OR role IN ('GYM_OWNER','SUPER_ADMIN'));

-- Capacity is enforced by the database, not by application logic (NFR-PRF-05)
CREATE TABLE bookings (
  id          uuid PRIMARY KEY,
  gym_id      uuid NOT NULL,
  location_id uuid NOT NULL,
  session_id  uuid NOT NULL REFERENCES sessions(id),
  user_id     uuid NOT NULL REFERENCES users(id),
  state       booking_state NOT NULL,     -- BOOKED | WAITLISTED | CANCELLED | ATTENDED | NO_SHOW
  position    int,                        -- waitlist order, FR-CLS-07
  -- One live booking per person per session. Double-tap cannot double-book.
  CONSTRAINT one_live_booking UNIQUE NULLS NOT DISTINCT (session_id, user_id, cancelled_at)
);

-- Credits can never go negative, whatever the application does (FR-BIL-10)
ALTER TABLE pt_packages ADD CONSTRAINT credits_non_negative
  CHECK (credits_remaining >= 0);

-- Offline replay is safe because the server deduplicates (ADR-06, FR-ATT-05)
CREATE TABLE attendance (
  id              uuid PRIMARY KEY,
  gym_id          uuid NOT NULL,
  location_id     uuid NOT NULL,
  user_id         uuid NOT NULL,
  occurred_at     timestamptz NOT NULL,
  method          checkin_method NOT NULL,   -- QR | GEOFENCE | MANUAL
  purpose         checkin_purpose NOT NULL,  -- SHIFT | PERSONAL_WORKOUT | BOTH  (FR-ATT-13)
  session_id      uuid REFERENCES sessions(id),
  recorded_by     uuid REFERENCES users(id), -- set when marked by staff
  deferred        boolean NOT NULL DEFAULT false,  -- synced from offline queue, FR-ATT-07
  idempotency_key text NOT NULL,
  CONSTRAINT attendance_idempotent UNIQUE (user_id, idempotency_key)
);
```

5.3 Row-Level Security — the implementation that matters

RLS is the load-bearing security control in this architecture. It is also easy to configure in a way that appears to work while protecting nothing. Three details decide whether it holds.

TWO DATABASE USERS, NOT ONE

> **A table owner bypasses RLS by default.** If the application connects as the user that owns the tables, every policy is silently ignored and the system looks correct in testing while enforcing nothing. Two separate roles are required, and every table must additionally be marked `FORCE ROW LEVEL SECURITY` so the rule applies even to ownership.

```
-- lb_migrator: owns the schema, runs Flyway, never serves a request
-- lb_app:      the application's runtime user, owns nothing, subject to every policy
ALTER TABLE members ENABLE ROW LEVEL SECURITY;
ALTER TABLE members FORCE  ROW LEVEL SECURITY;   -- applies to the owner too

CREATE POLICY gym_and_location_isolation ON members
  USING (
    current_setting('app.is_super_admin', true)::boolean IS TRUE
    OR (
      gym_id = current_setting('app.gym_id', true)::uuid
      AND (
        current_setting('app.gym_wide', true)::boolean IS TRUE   -- Gym Owner
        OR location_id = ANY(string_to_array(
             current_setting('app.location_ids', true), ',')::uuid[])
      )
    )
  );
```

CONTEXT MUST BE SET WITH `SET LOCAL`, INSIDE THE TRANSACTION

> **Plain `SET` would be a cross-tenant data leak.** Connections are pooled and reused. `SET` persists on the connection after the request finishes, so the next request — potentially serving a different gym — would inherit the previous tenant's context. `SET LOCAL` is scoped to the transaction and is discarded on commit or rollback, which is what makes pooling safe. This is not a style preference; it is the difference between isolation and a breach.

```
@Component
public class RlsContextInterceptor {

  @Transactional  // SET LOCAL requires an open transaction to be meaningful
  public void apply(AuthContext ctx) {
    jdbc.update("SET LOCAL app.gym_id = ?",         ctx.gymId());
    jdbc.update("SET LOCAL app.location_ids = ?",   String.join(",", ctx.locationIds()));
    jdbc.update("SET LOCAL app.gym_wide = ?",       ctx.isGymOwner());
    jdbc.update("SET LOCAL app.is_super_admin = ?", ctx.isSuperAdmin());
  }
}
```

A `@Transactional` aspect applies this at the start of every request-bound transaction. A request that somehow reaches the database without context set resolves `current_setting(…, true)` to `NULL`, the policy matches nothing, and queries return zero rows — the safe failure direction, and a loud one in testing.

SUPER ADMIN ACCESS IS DELIBERATE AND AUDITED

`app.is_super_admin` bypasses the policy, which `FR-ORG-09` requires for cross-gym oversight. It is set only when a Super Admin acts in an explicitly cross-gym context, never as an ambient default, and every such request is written to the audit trail. Note that this bypass grants *operational* visibility only, and never exceeds a Gym Owner's data tier. Purpose restrictions on health data are enforced separately at the application layer, since they are restrictions on purpose rather than on tenancy — see LB-ENG-1.0 §13 for the authoritative permission matrix.

TESTING

RLS cannot be verified against an in-memory database; it requires real Postgres. Testcontainers spins one up per suite. The standing test attempts, through every repository, to read another location's members, another gym's payments, and a sibling location's attendance — asserting empty results in each case. This suite *is* the evidence for `NFR-SEC-01a` and `NFR-SEC-01b`, and is the last thing that should ever be disabled to make a build pass.

5.4 Migrations

Flyway, versioned SQL, applied by `lb_migrator` at deploy time before the application starts accepting traffic. Every migration is forward-only; corrections are new migrations. Seed data — the exercise catalogue, muscle group taxonomy, default notification templates — ships as repeatable migrations so environments are reproducible rather than hand-assembled.

5.5 Indexing Strategy

Access pattern	Index
RLS filtering (every query)	Composite `(gym_id, location_id)` leading on every operational table. RLS predicates are evaluated per row, so these indexes are load-bearing rather than incidental.
Attendance history and streaks	`(user_id, occurred_at DESC)`; `(location_id, occurred_at DESC)` for footfall reporting.
Timetable browsing	`(location_id, starts_at)` with a partial index excluding cancelled sessions.
Expiry sweeps	Partial index on `(expires_at)` where status is active — keeps the nightly sweep cheap as history grows.
Job queue	`(state, run_after)` partial on pending, which is what makes `SKIP LOCKED` polling inexpensive.
Workout history	`(user_id, performed_at DESC)`; personal records derived by window function rather than stored redundantly.

At `NFR-SCL-03`'s stated 5 million attendance and 20 million workout set rows, no partitioning is required — Postgres handles these volumes comfortably on modest hardware. Monthly partitioning of `attendance` is the first move if it ever becomes necessary, and the schema is compatible with adding it later.

6. Security Architecture

6.1 Authentication Flow

1	**Client**	Submits mobile number. Server rate-limits per number and per device (`FR-AUTH-12`, `NFR-SEC-08`) before anything else happens.
2	**Twilio Verify**	Server requests a verification; Twilio generates, delivers and tracks the code. No OTP is generated or stored by us.
3	**Client**	Submits the code. Server checks it with Twilio, which enforces expiry and attempt limits.
4	**identity**	On success, loads the user's live role grants. A user in *Pending Verification* receives a restricted token permitting only profile view and correction (`FR-AUTH-09`).
5	**identity**	Issues a 15-minute access token carrying grants, plus a revocable refresh token stored server-side.
6	**Client**	Stores tokens in the device secure store (Keychain / Keystore) via `expo-secure-store` — never `AsyncStorage`.

6.2 Authorisation — role grants evaluated per location

`FR-AUTH-28` requires that effective authority be the union of an account's grants *evaluated per location*, never the maximum applied everywhere. A trainer who is also Location Admin at one site must not exercise admin authority at another. This is the model's subtlest rule and the easiest to get wrong, so it is expressed once, centrally:

```
public boolean can(AuthContext ctx, Permission p, LocationId at) {
    return ctx.grants().stream()
        .filter(g -> g.appliesAt(at))          // scope check FIRST — never skipped
        .anyMatch(g -> g.role().permits(p));
}
```

Authorisation is evaluated at the service layer, not in controllers, so no endpoint can bypass it by omission. RLS then provides the second, independent line: even a service method with a flawed check cannot return rows the database will not release. The two controls are deliberately redundant — a single mistake breaches neither.

6.3 Defence Summary

Concern	Control	Requirement
Cross-gym access	RLS policy on `gym_id`, enforced by Postgres on every query	`NFR-SEC-01`
Cross-location access	RLS policy on `location_id`, plus service-layer scope checks	`NFR-SEC-01a`
Privilege escalation	Grant creation validated against the granter's own scope; standing test suite over the delegation chain	`NFR-SEC-01b`, `FR-AUTH-18`
Credential theft	Argon2id password hashing; tokens in device secure storage; short access-token lifetime	`NFR-SEC-03/04`
QR replay	Rotating, time-limited, location-scoped signed payloads — a photographed code is useless elsewhere or later	`NFR-SEC-10`

Concern	Control	Requirement
Media exposure	Sensitive media uploaded as Cloudinary `authenticated` / `private` type; signed, time-limited URLs issued only after an authorisation check. Public assets separated deliberately — the default public type is never used for a receipt or identity document (ADR-15)	`NFR-SEC-07`
Brute force	Rate limiting per account and per device on auth, OTP issuance and check-in	`NFR-SEC-08`
Secrets	AWS Secrets Manager or SSM Parameter Store injected at runtime; never in the image or in source control	`NFR-SEC-09`
Dependency risk	Dependabot plus OWASP dependency-check in the CI pipeline, failing the build on high severity	`NFR-SEC-11`
Health data privacy	Application-layer purpose restriction independent of tenancy; access audited	`NFR-PRV-02/03`

7. Mobile Architecture

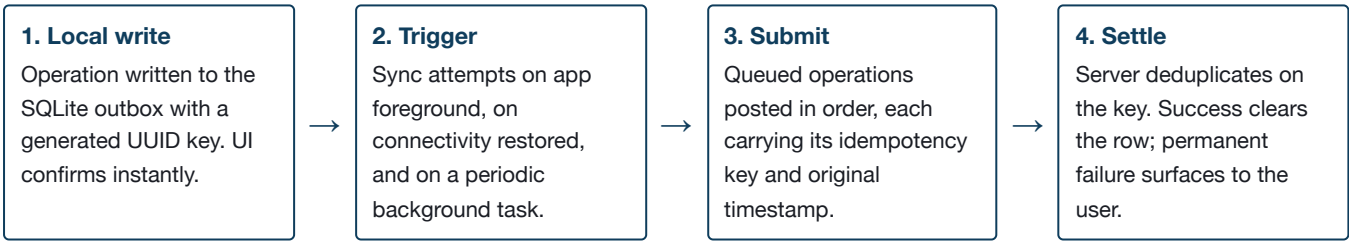
7.1 Structure

```
app/
├── (auth)/              // phone entry, OTP, pending-verification holding screen
├── (training)/          // EVERY role sees this — FR-PFP-01, FR-RPT-02
│   ├── today, workout/[id], progress, nutrition, bookings
├── (member)/            // plan, dues, receipts, feedback
├── (trainer)/           // roster, sessions, class attendance, plan authoring
├── (admin)/             // verifications, members, payments, timetable — location-scoped
├── (superadmin)/        // gym verification, activation, subscriptions
└── checkin/             // QR scan + geofence — the most-used screen in the app

lib/
├── api/                 // generated from OpenAPI — never hand-written
├── db/                  // expo-sqlite + Drizzle: outbox and cached reads
├── sync/                // background sync, retry, idempotency
└── auth/                // secure token storage, refresh, grant-aware navigation
```

Navigation is driven by the role grants in the token. The `(training)` group is mounted unconditionally for every account — the routing layer's expression of the SRS dual-facet model. An account holding several grants sees several groups simultaneously, each labelled, with no context switch (`FR-AUTH-27`).

7.2 Offline Sync



Only check-in and workout set logging are queued. The reason is structural rather than a matter of effort: as noted in ADR-06, both are non-contending — a check-in is an append-only fact, and a workout log has exactly one author. Neither requires arbitration, so no conflict-resolution strategy is needed. Bookings, payments and plan changes contend for shared resources and are therefore always online, arbitrated by the database.

7.3 Idempotency — what makes the queue safe

The retry problem is that a network failure is ambiguous: the client cannot tell a request that never arrived from one that succeeded but whose response was lost. Retrying risks duplication; not retrying risks loss.

Idempotency keys resolve it. The client generates a UUID per logical operation and reuses it across every retry. The server stores it against the resulting record under a unique constraint, so a replay returns the original outcome rather than creating a second. This is what lets the client retry freely, and it is simultaneously how `FR-ATT-05`'s duplicate-rejection requirement is met under offline replay — one mechanism satisfying a correctness requirement and an availability requirement at once.

7.4 Push Notifications

An Expo push token is registered per device on sign-in and invalidated on sign-out. Dispatch runs through the job queue (ADR-09), so a slow or failing push provider never blocks a request. Delivery receipts are polled from

Expo and recorded per notification, satisfying `FR-NTF-06`. Quiet hours (`FR-NTF-04`) and de-duplication (`FR-NTF-07`) are applied server-side at dispatch, never on the device — the device cannot be trusted to suppress what it has already received.

8. Background Processing

All deferred work runs through one queue table. Jobs are enqueued in the same transaction as the business change that triggers them, so a rollback removes the job too — no orphaned notifications for a booking that never happened.

```
-- One worker claims a batch; concurrent workers never collide
SELECT * FROM job_queue
 WHERE state = 'PENDING' AND run_after <= now()
 ORDER BY priority DESC, run_after
 FOR UPDATE SKIP LOCKED     -- the whole trick: locked rows are skipped, not waited on
 LIMIT 20;
```

Job	Trigger and purpose	Phase
`NotificationDispatch`	Any notifiable event; renders the template and sends via Expo Push	1
`StreakRecalc`	After a check-in; recomputes streak off the request path	1
`MembershipExpirySweep`	Nightly; flags expiring memberships and raises reminders (`FR-BIL-06`)	1
`SubscriptionSweep`	Nightly; gym subscription expiry, grace period, lapse transitions (`FR-SUB-06`)	1
`SessionGeneration`	Nightly; materialises sessions from recurring timetables over the rolling horizon (`FR-CLS-03`)	3
`WaitlistPromotion`	On booking release; promotes and notifies the first waitlisted member (`FR-CLS-08`)	3
`SessionReminder`	Scheduled ahead of each session; also releases the online meeting link (`FR-ONL-03`)	3
`WhatsAppSend`	Outbound templated message with retry and per-member daily cap (`FR-WA-10`)	4
`HabitualTimeNudge`	Derives each member's usual attendance time and prompts on days they have not come (`FR-WA-06`)	4
`AiGeneration`	Calls the provider, stores the draft for human approval — never auto-publishes (`FR-AI-03`)	4
`ReportMaterialise`	Pre-computes heavy reports, satisfying the async escape in `NFR-PRF-06`	4

9. Integration Architecture

Per `PRN-05`, every integration sits behind an interface owned by our code, with a defined degradation path. No external failure may cascade into core operations (`NFR-AVL-04`, `CM-03`).

Service	Used for	Degradation when unavailable	Phase
Twilio Verify	Phone and email OTP	Email-and-password sign-in remains available. New registration is blocked — the one genuinely hard dependency in Phase 1, which is why DLT registration is on the critical path.	1
Expo Push	Notifications	Notifications persist in the in-app inbox and are seen on next open; nothing is lost, only delayed.	1
Cloudinary	Media, receipts	Uploads queue client-side and retry; existing media is briefly unavailable. No write path blocks — media is never on the critical path of a check-in, booking or payment.	1
Zoom / Meet	Online class delivery	None — links are plain text entered by staff. No API integration exists to fail (`SW-04`).	3
WhatsApp	Outbound messaging	Falls back to push notification per `FR-WA-09`; message rows retain failure status for review.	4
AI provider	Drafts and insights	All manual authoring paths remain fully functional, as `FR-AI-08` requires. Interface abstracted so the provider is replaceable — the predecessor used Mistral; the choice is re-evaluated at Phase 4 on cost and quality.	4

10. Deployment, Environments and CI/CD

10.1 Topology

DEVELOPMENT TOPOLOGY — CURRENT

EC2 · ap-southeast-1
`t4g.small/medium` (ARM). Nginx + Let's Encrypt → Docker container running Spring Boot. Elastic IP.

→

Neon · ap-southeast-1
Postgres 16, pooled endpoint, scale-to-zero disabled. **Same region — this is the point of ADR-07.**

Cloudinary
Media, CDN, transformation

S3
Database backups only

Secrets Manager
Runtime credential injection

CloudWatch + Sentry
Logs, metrics, errors

PRODUCTION TARGET — ON FIRST PAYING GYM

ALB
TLS, health checks

→

EC2 ×2 · ap-south-1
Mumbai, stateless, rolling deploys

→

RDS Postgres · ap-south-1
Multi-AZ, automated backups, PITR

Moving to Mumbai for both application and database gives Indian users the lowest achievable latency. Because Neon and RDS are both stock PostgreSQL 16, the migration is a dump, a restore and a connection string — no application change, which is what makes deferring it safe.

10.2 Environments

Environment	Purpose and configuration
Local	Docker Compose with Postgres 16, matching production's RLS role separation so isolation bugs surface on a laptop rather than in production.
Development	The EC2 and Neon topology above. Seeded with representative data: several gyms, several locations each, so isolation is exercised continuously rather than at review time.
Production	Introduced at first paying gym. Separate AWS account, separate secrets, no shared infrastructure with development.

> **Seed development with more than one gym, from day one.** The most dangerous defect class in this system is cross-tenant leakage (`RSK-06`), and it is invisible when only one gym exists in the database. A development environment holding three gyms with two locations each makes isolation failures show up during ordinary feature work, which is by far the cheapest place to find them.

10.3 Pipelines

Pipeline	Steps
Backend PR	Compile → unit tests → Testcontainers integration tests including the RLS isolation suite → OWASP dependency-check → Modulith boundary verification. All must pass to merge.
Backend deploy	Build multi-arch image → push to ECR → SSH deploy to EC2 → Flyway migrations run as `lb_migrator` → rolling container restart → health check → automatic rollback on failure.
Mobile build	EAS Build for iOS and Android → internal distribution for testing → EAS Submit to App Store Connect and Google Play.
Mobile hotfix	EAS Update pushes JavaScript-only changes over the air, bypassing store review entirely — the difference between a same-hour fix and a three-day wait.

11. Observability, Backup and Recovery

Concern	Approach	Requirement
Structured logging	JSON to stdout, collected by CloudWatch. Every log line carries a correlation ID, and — critically — the acting user, gym and location, so a failed check-in can be reconstructed without reproducing it.	`NFR-MNT-02`
Error tracking	Sentry on both backend and mobile, with release tagging so a spike is attributable to a specific deploy or OTA update.	`NFR-MNT-02`
Metrics	Micrometer via Spring Actuator: request latency percentiles, job queue depth and age, connection pool saturation, check-in throughput.	`NFR-PRF-01`
Health checks	`/health` for the load balancer; `/health/deep` verifying database and Cloudinary reachability for alerting.	`NFR-AVL-01`
Alerting	Error rate, p95 latency breach, job queue age exceeding threshold, failed deploy, backup failure.	`NFR-AVL-01`
Backups	Neon provides point-in-time restore during development; RDS automated backups with PITR in production. Additionally, a nightly `pg_dump` to a separate S3 bucket with versioning and a lifecycle policy — because a backup in the same account as the database is only half a backup.	`NFR-AVL-02`
Media backups	Cloudinary holds the only copy of uploaded media, and a database restore would therefore recover the `public_id` references but not the images themselves. Accepted during development; a periodic export of sensitive media into the same S3 backup bucket is required before any pilot (`ARK-10`).	`NFR-AVL-02`
Restore drills	A quarterly, documented, actually-executed restore into a scratch environment, timed against the 4-hour RTO. **An untested backup is a hypothesis, not a recovery plan** — and this is the requirement most often quietly skipped, which is exactly why it is named here.	`NFR-AVL-03`

12. Performance and Scaling Strategy

The SRS targets — 50 gyms, 200 locations, 100,000 members, 500 concurrent users, 5 million attendance rows — are modest for this stack. A single well-sized instance and a single Postgres database handle them with room to spare. The interesting characteristic is not volume but *shape*: load concentrates violently into the 6–8am and 6–9pm windows (`NFR-SCL-04`), so capacity must be sized against peak and not average.

Pressure	Response, in order
Check-in bursts	Java 21 virtual threads absorb concurrent blocking calls without reactive rewriting. Derived work (streaks, notifications) is already off the request path via the job queue.
Slow dashboards	Indexes first; then materialised views refreshed by job for the heaviest reports; then the async escape `NFR-PRF-06` already permits.
Database contention	Connection pool tuning; read replicas for reporting once write and read patterns genuinely diverge. Not before.
Instance saturation	Vertical scaling first (a one-line change). Then ALB plus a second instance — the application is stateless and the queue already tolerates concurrent consumers, so nothing else changes.
Storage growth	Cloudinary credits are the binding constraint on media, not disk. Cap upload dimensions at the point of upload and rely on derived renditions rather than storing multiple sizes. Monthly partitioning of `attendance` if row volume ever demands it.

> **Measure before optimising.** Every scaling move above is a response to an observed metric, not a precaution. The metrics in §11 exist to make that possible. The architecture's most valuable performance property is that none of these responses requires re-architecting — each is a configuration or infrastructure change against the same codebase.

13. Indicative Cost Model

Approximate monthly figures at development scale, in INR. Directional only — verify against current pricing.

Item	Development	Notes
EC2 `t4g.small`	₹1,200–1,600	2 vCPU / 2GB ARM. Step to `t4g.medium` (4GB, ~₹2,800) if the JVM proves tight — likely once background workers run alongside the API.
Neon Postgres	₹0 – 1,700	Free tier initially, then Launch (~$19). Disable scale-to-zero once latency is being measured.
Cloudinary	₹0	Free tier: 25 credits/month, consumed jointly by storage, bandwidth *and* transformations. Adequate for development; exercise media and receipts at real volumes will exceed it, at which point the paid tier starts around $89/month (~₹7,400) — the trigger to weigh S3 plus a thumbnail pipeline against it.
S3 (backups only)	₹100–300	Nightly database dumps. Media no longer lives here.
Twilio Verify	Usage-based	Roughly ₹0.50–3 per verification into India. A thousand sign-ins and logins a month is a small line item; free credits cover early development.
Sentry, CloudWatch	₹0–500	Free tiers are adequate at this stage.
Expo EAS	₹0–8,300	Free tier covers occasional builds; the Production plan (~$99) matters only once release cadence increases.
Running total	**₹1,400–5,000**	Excluding EAS Production and Twilio usage.
Apple Developer	₹8,300/year	$99 annually — mandatory for App Store distribution.
Google Play	₹2,100 once	$25 one-time registration.

Production on RDS Multi-AZ with an ALB and two instances lands around ₹15,000–25,000/month. At 50 gyms that is a negligible fraction of subscription revenue, so the step up is a commercial non-event when it arrives.

14. Build Sequence by Phase

Phase	Architectural focus	What gets built, and why in this order
0	Foundations	Repository layout, Modulith skeleton, Docker Compose, Flyway with the two-role RLS setup, CI pipeline, Expo project scaffold. **The RLS isolation test suite is written before the first feature** — retrofitting isolation tests onto existing code is how gaps survive.
1	Identity, tenancy, operations	`platform`, `identity`, `org`, `people`, `billing`, `attendance`, plus the `training` module's personal-profile core. Twilio and DLT registration must start on day one of this phase, since sign-in cannot work without it.
2	Coaching	The rest of `training` — exercises, routines, set logging, nutrition, progress — plus PT packages in `billing` and feedback in `engagement`. The offline outbox ships here, once workout logging exists to use it.
3	Scheduling	`scheduling` in full: timetables, session generation, bookings, capacity, waitlists, online links. The most concurrency-sensitive module, and the strongest vindication of the single-transaction monolith.
4	Intelligence and reach	`insights` and the remainder of `engagement`: reporting, exports, WhatsApp, AI. Materialised views introduced here if dashboard latency requires them.
5	Scale	Self-service onboarding, per-gym branding, and the RDS Mumbai migration per ADR-07's revisit trigger.

15. Risks and Open Decisions

ID	Risk	Impact and mitigation
ARK-01	RLS misconfiguration silently disables isolation	The highest-severity technical risk here. Connecting as the table owner, or using `SET` instead of `SET LOCAL`, produces a system that passes ordinary testing while protecting nothing. Mitigated by the two-role setup, `FORCE ROW LEVEL SECURITY`, multi-gym seed data in development, and a standing isolation suite that must never be skipped to green a build.
ARK-02	DLT registration delays Phase 1	No OTP means no sign-in means no usable system. One to two weeks, outside our control. Start immediately; keep email-and-password sign-in fully functional as a hedge.
ARK-03	Cross-region database latency	Addressed by co-location (ADR-07), but only if actually configured that way. Assert it: alert on p95 database round-trip time exceeding 5ms.
ARK-04	Neon cold starts breach latency targets	Scale-to-zero adds 500ms to several seconds on the first request after idle. Disable it wherever latency is measured; be aware the free tier may not permit disabling.
ARK-05	App Store rejection over health data	Progress photographs, health screening and location access all attract scrutiny. Mitigate with a published privacy policy, in-context permission rationale, and a store listing consistent with `CON-07`'s position that this is fitness guidance, not medical advice.
ARK-06	Single EC2 instance is a single point of failure	Accepted during development. Mitigated by automated off-instance backups and a documented rebuild procedure; resolved by the ALB and second instance at production.
ARK-07	Modulith boundaries erode under delivery pressure	Boundaries decay silently and are expensive to restore. Mitigated by making Modulith verification a build-failing CI step rather than a convention.
ARK-08	PgBouncer transaction mode breaks prepared statements	Neon's pooled endpoint requires `prepareThreshold=0` on the JDBC URL. Symptom is intermittent, confusing errors under load rather than a clean failure — worth configuring correctly at the outset.
ARK-09	Progress photographs uploaded to Cloudinary's public delivery type	The default upload type yields a permanent public URL. One sensitive image uploaded through the default path is a privacy breach that cannot be undone by deleting the record, because the URL may already be cached or shared. Mitigated by routing every upload through `MediaService` with sensitivity as a required argument rather than an option, and by a test asserting that no sensitive asset is ever created with the public type. Verify plan support for expiring authenticated delivery before Phase 2 (ADR-15).
ARK-10	Media has no second copy	Cloudinary holds the only copy of every uploaded asset. Unlike the database, there is no nightly dump. Accepted during development; before a pilot, add a periodic export of sensitive media to the same S3 backup bucket used for database dumps.

15.1 Open Decisions

ID	Decision	Needed by
AOP-01	Team size and Phase 1 target date — `OPN-01` in the SRS, still open	Affects nothing structural here; the architecture is deliberately viable for one developer. But it determines whether the phase sequence in §14 is realistic.

ID	Decision	Needed by
AOP-02	A person belonging to more than one gym — `OPN-04`	The `role_grants` model can express cross-gym grants, but RLS context assumes a single active `gym_id` per request. If cross-gym users are permitted, a gym-switching mechanism is needed in the token and context. **Decide before the identity module is built** — it is cheap now and expensive later.
AOP-03	AI provider for Phase 4	Not needed until Phase 4. The interface is abstracted, so the decision can wait for evidence on cost and output quality.
AOP-04	Web console timing	The SRS requires it for admin work. Whether it ships in Phase 1 alongside mobile, or follows in Phase 2, is a capacity decision rather than an architectural one — the backend is identical either way.

Appendix A — Requirement to Architecture Mapping

The SRS requirements that most constrain the design, and where each is satisfied.

Requirement	Demand	Where satisfied
NFR-SEC-01/01a	Gym and location isolation	RLS policies (§5.3), service-layer scope checks (§6.2), standing Testcontainers suite
NFR-SEC-01b	No privilege escalation	Grant validation against granter scope; delegation-chain test suite
NFR-PRF-05	No overselling a session	Single transaction with row lock plus unique constraint (§4.2, §5.2)
NFR-PRF-02	Check-in under 2s on 3G	Local-first write (§7.2); server path measured at 7–17ms (§4.4)
FR-ATT-05/07	Duplicate rejection; offline queue	Idempotency keys under a unique constraint — one mechanism, both requirements (§7.3)
FR-AUTH-26/28	Trainer + Location Admin; union per location	`role_grants` with array scope (§5.2); central `can()` evaluation (§6.2)
FR-PFP-01	Personal profile for every role	`training` module takes no role parameter and branches on none (§4.3); `(training)` route group mounted unconditionally (§7.1)
FR-ATT-13	Shift versus personal workout	`purpose` column on attendance (§5.2); reports filter on it (§8)
FR-SUB-*	Gym subscription lifecycle	`platform` module; `SubscriptionSweep` job (§8)
NFR-SEC-07	Media not publicly reachable	Cloudinary authenticated delivery type with signed, time-limited URLs issued post-authorisation; public and sensitive media deliberately separated (ADR-15). Progress photographs are out of scope, removing the most sensitive media class entirely
NFR-SEC-10	QR replay resistance	Rotating, time-limited, location-scoped signed payloads (§6.3)
NFR-AVL-03	Tested restores	Quarterly executed restore drills, timed against RTO (§11)
NFR-MNT-04	Old app builds keep working	Versioned API path plus OpenAPI-generated clients (ADR-13)

Appendix B — SRS Amendments Required

Decisions taken here that contradict LB-SRS-1.1 and require it to be revised to v1.2. Recorded rather than applied silently, so the SRS remains the authority on requirements.

Requirement	Current wording	Required amendment
FR-EXR-05	"The muscle group taxonomy and seeded exercise data from limitbeyond2.0 shall be carried into the new catalogue."	**Revise.** Per ADR-11, nothing is migrated. The catalogue is authored fresh against the richer model — primary and secondary muscle groups, equipment, difficulty, instructions and media — because the predecessor's records lack those fields and a port would yield incomplete rows requiring manual completion regardless.
OPN-02	Open: migration from limitbeyond2.0 undecided.	**Close.** Clean start; nothing migrated. The predecessor continues running until cutover.
OPN-03	Already closed in v1.1 (gateway withdrawn).	No change. Confirmed by this architecture — no payment integration exists at any tier.
OPN-04	Open: a person belonging to more than one gym.	**Escalate.** Now a build-blocking decision, not merely an open question — RLS context assumes one active gym per request (`A0P-02`). Must be settled before the `identity` module is implemented.

> **What this document does not decide.** API endpoint signatures, full table definitions, screen designs and test plans belong to implementation, not architecture. What is fixed here is the set of choices that would be expensive to reverse later: the language, the database, how isolation is enforced, the shape of the codebase, and where state lives. Everything else is deliberately left to be decided with more information than we have today.

— END OF DOCUMENT —